

DODGER: Safety-Guided Reinforcement Learning for Robot Navigation Among Dynamic Obstacles

Sanghyuk Park^{*1}, Kwanwoo Lee^{*1}, Taekyung Kim^{*2}, Seohyeon Lim³, and Yisoo Lee^{†4}

Abstract—Robots operating in human-centered environments must safely navigate among multiple dynamic obstacles to avoid collisions with people and surrounding infrastructure. Control barrier functions (CBFs) provide an effective mechanism for safety filtering, and recent CBF-based reinforcement learning (RL) methods embed such safety information into learned policies. However, executing only safety-filtered actions during training can restrict policy exploration, a limitation that becomes particularly consequential in dynamic scenes where safety depends on relative robot-obstacle motion. We propose DODGER, a safety-guided RL framework that directly executes policy-generated actions to drive training rollouts while using CBF-filtered references and constraint violations to shape the policy toward collision-avoidance behavior. We evaluate DODGER through a Dubins-car safety analysis and demonstrate goal-directed navigation among multiple dynamic obstacles in full-order humanoid simulation and real-world humanoid experiments using LiDAR-based perception, without a runtime safety filter. [Project Page]¹ [Code]² [Web Demo]

I. INTRODUCTION

Learning-based control has enabled agile and adaptive behaviors across diverse robotic platforms, from aerial vehicles to legged robots [1], [2]. As robots increasingly operate in dynamic, human-centered environments, they must navigate toward task goals while safely interacting with moving obstacles. This requirement is particularly relevant to humanoid robots operating in human environments, where navigation often involves close interactions with moving people. For humanoids, dynamic-obstacle avoidance is typically handled through a low-dimensional navigation interface, while a separate whole-body controller realizes the commanded motion. The high-level navigation policy must therefore select goal-directed commands while accounting for the relative motion of surrounding obstacles.

Dynamic-obstacle avoidance is commonly addressed through online planning or safety filtering. MPC optimizes over a finite horizon using robot dynamics and predicted

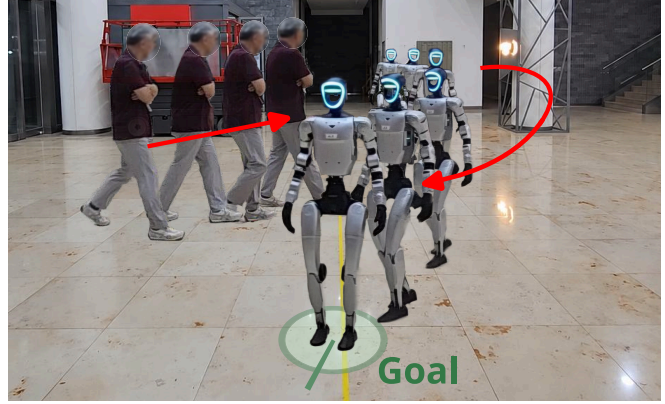


Fig. 1: A humanoid avoids a moving pedestrian while navigating toward a goal using the DODGER policy. The policy is trained with CBF-based safety guidance and deployed without a runtime safety filter.

obstacle motion, but its computational cost increases with the planning horizon and the number of obstacles [3], [4]. CBF-based safety filters instead modify nominal commands online to satisfy prescribed safety constraints with minimal intervention [5]. For hierarchical humanoid control, however, such constraints are typically formulated on a reduced-order navigation model, while the commanded motion is realized by the full-body system. Model mismatch and tracking error can therefore create a gap between the safety condition imposed on the high-level command and the realized robot motion [6]. This motivates learning a navigation policy that is itself informed by the safety structure, rather than relying on online correction alone.

In Reinforcement Learning (RL), safety information is commonly incorporated through reward shaping or safety-filtered interaction [7]. Reward-based methods penalize constraint violations [8], but their effectiveness can depend strongly on the design and weighting of the penalty terms [9]. Safety-filtering methods instead modify policy actions before environment interaction to satisfy safety constraints [10]–[12]. CBF-RL combines such filtering with a reward that penalizes deviations between nominal and filtered actions [13]. However, because the filtered action is executed during training, the collected experience follows the state-action distribution induced by the safety filter rather than the policy itself. This can limit exposure to states arising from the policy’s own actions, motivating a formulation that preserves policy-generated interaction while using CBF information only to guide learning.

To address these limitations, we propose **DODGER**, Dynamic Obstacle Avoidance with **DPCBF**-Guided

^{*}Equal contribution; [†]Corresponding author.

This work was supported by the National Research Foundation of Korea (NRF) grant funded by the Korea government (MSIT) (RS-2026-25540546).

¹Sanghyuk Park and Kwanwoo Lee are with the Department of Intelligence and Information, Seoul National University, Republic of Korea. sang0823, kwlee365@snu.ac.kr

²Taekyung Kim is with the Department of Robotics, University of Michigan, Ann Arbor, MI, USA. taekyung@umich.edu

³Seohyeon Lim is with the Department of Mechanical Engineering, Yonsei University, Republic of Korea. joan0219@yonsei.ac.kr

⁴Yisoo Lee is with the Center for Humanoid Research, Korea Institute of Science and Technology (KIST), Seoul, Republic of Korea. yisoo.lee@kist.re.kr

¹Project page: <https://psh0823.github.io/dodger-homepage>

²Code: <https://github.com/psh0823/dodger>

Exploration in Reinforcement Learning, a safety-guided RL framework for goal-directed robot navigation among multiple dynamic obstacles. DODGER uses the Dynamic Parabolic CBF (DPCBF) [14] to capture safety constraints induced by relative robot-obstacle motion, and integrates it with graph-based scene encoding and our safety-guided RL formulation. Rather than executing safety-filtered actions during training, DODGER directly rolls out policy-generated actions while using the filtered reference and constraint violations to guide policy updates.

The main contributions of this work are:

- We introduce a safety-guided RL training formulation in which policy-generated actions directly drive environment interaction, while CBF-filtered references and constraint evaluations are used only to guide policy learning. This allows the policy to learn from the consequences of its own actions without relying on safety-filtered rollouts.
- We introduce a constraint-aware return formulation that uses CBF and input-constraint violations to reduce positive returns, while a two-head critic preserves task-specific negative penalties.
- We develop DODGER for navigation among multiple dynamic obstacles using DPCBF-based safety guidance and graph-based scene encoding. We evaluate how the learned critic captures unsafe regions in a Dubins-car case study, and demonstrate goal-directed multi-obstacle navigation in full-order humanoid simulation and real-world deployment on a Unitree G1 (Fig. 1).

II. PRELIMINARIES

A. Control Barrier Functions

Consider a continuous-time control-affine system

$$\dot{\mathbf{x}} = f(\mathbf{x}) + g(\mathbf{x})\mathbf{u}, \quad (1)$$

where $\mathbf{x} \in \mathcal{X} \subseteq \mathbb{R}^n$ denotes the state and $\mathbf{u} \in \mathcal{U} \subset \mathbb{R}^m$ denotes the control input. The functions $f : \mathcal{X} \rightarrow \mathbb{R}^n$ and $g : \mathcal{X} \rightarrow \mathbb{R}^{n \times m}$ are assumed to be locally Lipschitz continuous.

Let $h : \mathcal{X} \rightarrow \mathbb{R}$ be a continuously differentiable function. The set $\mathcal{C}$ is defined as the zero superlevel set of h : $\mathcal{C} = \{\mathbf{x} \in \mathcal{X} \mid h(\mathbf{x}) \geq 0\}$. A function h is a control barrier function if there exists an extended class- $\mathcal{K}_\infty$ function $\alpha(\cdot) : \mathbb{R} \rightarrow \mathbb{R}$ such that

$$\sup_{\mathbf{u} \in \mathcal{U}} [L_f h(\mathbf{x}) + L_g h(\mathbf{x})\mathbf{u} + \alpha(h(\mathbf{x}))] \geq 0, \quad \forall \mathbf{x} \in \mathcal{X},$$

where $L_f h$ and $L_g h$ denote the Lie derivatives of h with respect to f and g , respectively. Under the standard regularity condition $\nabla h(\mathbf{x}) \neq 0$ for all $\mathbf{x} \in \partial\mathcal{C}$, any locally Lipschitz feedback controller $\mathbf{u} = \mathbf{k}(\mathbf{x})$ satisfying $L_f h(\mathbf{x}) + L_g h(\mathbf{x})\mathbf{k}(\mathbf{x}) + \alpha(h(\mathbf{x})) \geq 0$ for all $\mathbf{x} \in \mathcal{X}$ renders $\mathcal{C}$ forward invariant for the closed-loop system [15]. In practice, this constraint is commonly enforced through a CBF-based Quadratic Program (CBF-QP) that minimally modifies a nominal control input.

B. CBF-RL

CBF-RL [13] integrates a CBF-QP safety filter with RL during training and uses a reward that encourages the actor output to remain close to the filtered action. In its humanoid locomotion experiments, the CBF is constructed on a reduced-order model while the policy is trained with the full-body dynamics. The resulting policy is deployed without solving the CBF-QP online and empirically exhibits fewer constraint violations.

However, two limitations remain. First, because CBF-RL executes the safety-filtered action during training, the collected experience follows the state-action distribution induced by the safety filter rather than that of the unfiltered policy [13]. This can restrict exposure to states that would otherwise be encountered by the unfiltered policy. Second, extending this framework to dynamic-obstacle navigation requires a safety representation that explicitly accounts for the relative motion between the robot and surrounding obstacles. The reported CBF-RL experiments focus on static environments, whereas safety in dynamic scenes depends jointly on the relative position and velocity of surrounding obstacles. These observations motivate preserving environment interaction under the policy's own actions while incorporating a barrier formulation tailored to dynamic obstacle interactions.

III. METHODOLOGY

To address these objectives, DODGER couples policy-generated environment interaction with Dynamic Parabolic CBF (DPCBF)-derived safety guidance within a hierarchical navigation framework. The DPCBF is formulated on a reduced-order navigation model and used during training to provide a safety-filtered reference action and quantify constraint violations, while the actor action drives the environment rollout. We first present the DPCBF formulation, followed by the overall DODGER architecture and the proposed learning framework.

A. Dynamic Parabolic CBF

The DPCBF constructs a state-dependent parabolic safety boundary from the relative position and velocity between the robot and surrounding obstacles. Its velocity-dependent formulation provides a first-order safety constraint for the reduced-order dynamics while adapting the boundary to dynamic obstacle interactions.

1) *Reduced-Order Dynamics*: We formulate navigation using a planar reduced-order model that captures translational motion in the forward and lateral directions of a body-fixed frame attached to the floating base, together with its heading motion. This representation provides a low-dimensional interface for constructing the CBF constraints independently of the full-order robot dynamics.

The state vector is given by $\mathbf{x} = [x, y, \phi, v_f, v_l]^\top$, where (x, y) denote the planar position of the floating base, ϕ denotes its heading angle, and v_f and v_l denote the forward and lateral velocities expressed in the body frame, respectively.

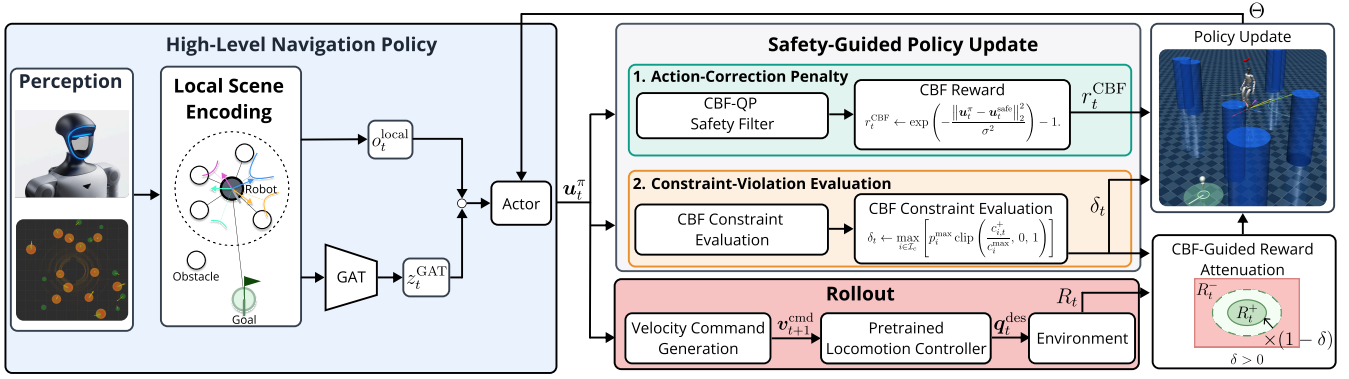


Fig. 2: Overview of DODGER. The high-level policy combines local robot-goal information with a graph-based representation of surrounding obstacles to generate navigation commands. During training, the policy action directly drives the environment rollout, while the CBF-filtered reference and constraint violations guide learning. The resulting navigation command is executed by a fixed pretrained locomotion controller.

The reduced-order dynamics are given by

$$\begin{bmatrix} \dot{x} \\ \dot{y} \\ \dot{\phi} \\ \dot{v}_f \\ \dot{v}_l \end{bmatrix} = \begin{bmatrix} v_f \cos \phi - v_l \sin \phi \\ v_f \sin \phi + v_l \cos \phi \\ 0 \\ 0 \\ 0 \end{bmatrix} + \begin{bmatrix} 0 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 1 \\ 1 & 0 & 0 \\ 0 & 1 & 0 \end{bmatrix} \begin{bmatrix} a_f \\ a_l \\ \omega \end{bmatrix}, \quad (2)$$

where the control input is $\mathbf{u} = [a_f, a_l, \omega]^\top$. Here, a_f and a_l denote the forward and lateral accelerations, respectively, and ω denotes the yaw angular velocity.

2) *DPCBF Formulation*: We consider N_{obs} detected dynamic obstacles, with the j -th obstacle state

$$\mathbf{x}_{\text{obs}}^j = [x_{\text{obs}}^j, y_{\text{obs}}^j, \phi_{\text{obs}}^j, v_{\text{obs}}^j]^\top, \quad (3)$$

where $(x_{\text{obs}}^j, y_{\text{obs}}^j)$, ϕ_{obs}^j , and v_{obs}^j denote its center, heading, and forward speed, respectively. Each obstacle is modeled with constant planar velocity during barrier evaluation, i.e., $\dot{\phi}_{\text{obs}}^j = \dot{v}_{\text{obs}}^j = 0$ [14]. The superscript j and explicit obstacle-state arguments are omitted below. All Lie derivatives are evaluated along the joint robot-obstacle dynamics.

Let $\mathbf{p}_{\text{rob}} = [x, y]^\top$ and $\mathbf{p}_{\text{obs}} = [x_{\text{obs}}, y_{\text{obs}}]^\top$. The relative position and velocity are $\mathbf{p}_{\text{rel}} = \mathbf{p}_{\text{rob}} - \mathbf{p}_{\text{obs}}$ and $\mathbf{v}_{\text{rel}} = \mathbf{v}_{\text{rob}} - \mathbf{v}_{\text{obs}}$, where $\mathbf{v}_{\text{rob}} = [\dot{x}, \dot{y}]^\top$ and $\mathbf{v}_{\text{obs}} = v_{\text{obs}} [\cos \phi_{\text{obs}}, \sin \phi_{\text{obs}}]^\top$. Following DPCBF [14], $\mathbf{v}_{\text{rel}}$ is expressed in a line-of-sight (LoS) frame whose $\tilde{x}$ -axis is aligned with $\mathbf{p}_{\text{rel}}$, yielding $\tilde{\mathbf{v}}_{\text{rel}} = [\tilde{v}_{\text{rel},x}, \tilde{v}_{\text{rel},y}]^\top$. We approximate the planar footprints of the robot and obstacle by circles with radii r_{rob} and r_{obs} , respectively, and define the combined safety radius $r = r_{\text{rob}} + r_{\text{obs}}$ and, for $\|\mathbf{p}_{\text{rel}}\| > r$, the distance-dependent margin $d(\mathbf{x}) = \sqrt{\|\mathbf{p}_{\text{rel}}\|^2 - r^2}$. The DPCBF parameters are $\lambda(\mathbf{x}) = k_\lambda d(\mathbf{x}) / \|\mathbf{v}_{\text{rel}}\|$ and $\mu(\mathbf{x}) = k_\mu d(\mathbf{x})$, where $k_\lambda, k_\mu > 0$. In implementation, $\|\mathbf{v}_{\text{rel}}\|$ is lower-bounded by a small $\epsilon_v > 0$ to avoid numerical singularity. The resulting DPCBF is

$$h(\mathbf{x}, \mathbf{x}_{\text{obs}}) = \tilde{v}_{\text{rel},x} + \lambda(\mathbf{x}) \tilde{v}_{\text{rel},y}^2 + \mu(\mathbf{x}). \quad (4)$$

The parameters $\lambda(\mathbf{x})$ and $\mu(\mathbf{x})$ adapt the parabolic boundary to the relative robot-obstacle state: $\lambda(\mathbf{x})$ controls its curvature, while $\mu(\mathbf{x})$ shifts its vertex along the $\tilde{v}_{\text{rel},x}$ axis. The boundary becomes less restrictive for more distant obstacles or smaller relative speeds, and more conservative as the obstacle approaches or the relative speed increases.

B. DODGER Overview

As illustrated in Fig. 2, DODGER couples a high-level navigation policy with a pretrained low-level policy. The pretrained low-level policy is kept fixed during high-level DODGER training. At each time step t , the high-level navigation policy receives a local observation defined as

$$\mathbf{o}_t^{\text{local}} = [(\mathbf{p}_{g,t}^b)^\top, \sin e_{\phi,t}, \cos e_{\phi,t}, (\mathbf{v}_t^b)^\top, \dot{\phi}_t, (\mathbf{v}_t^{\text{cmd}})^\top, (\mathbf{u}_{t-1}^\pi)^\top]^\top, \quad (5)$$

where superscript b denotes body-frame coordinates, $\mathbf{p}_{g,t}^b \in \mathbb{R}^2$ is the relative goal position, and $e_{\phi,t} = \text{wrap}(\phi_g - \phi_t)$ is the heading error for goal heading ϕ_g , wrapped to $[-\pi, \pi)$. The measured planar velocity and yaw rate are $\mathbf{v}_t^b = [v_{f,t}, v_{l,t}]^\top$ and $\dot{\phi}_t$. The current velocity command is $\mathbf{v}_t^{\text{cmd}} = [v_{f,t}^{\text{cmd}}, v_{l,t}^{\text{cmd}}, \omega_t^{\text{cmd}}]^\top$, and $\mathbf{u}_{t-1}^\pi \in \mathbb{R}^3$ is the previous high-level actor action.

To accommodate a varying number of surrounding obstacles, DODGER represents the robot, goal, and detected obstacles as a graph $\mathcal{G}_t = (\mathcal{V}_t, \mathcal{E}_t)$, with node set $\mathcal{V}_t$ and edge set $\mathcal{E}_t$. Following [16], we use robot-centered attention: the robot node serves as the query over the goal and all detected obstacle nodes, with no message passing among non-robot nodes. Each node corresponds to the robot, goal, or a detected obstacle, with a one-hot encoding indicating its node type. The edge features encode the relative position and velocity between connected nodes, together with the minimum distance between their boundaries. A graph attention encoder (GAT) aggregates these features into a robot-node embedding

$$\mathbf{z}_t^{\text{GAT}} = \Phi_\psi^{\text{GAT}}(\mathcal{G}_t), \quad (6)$$

where Φ_ψ^{GAT} denotes the graph attention encoder parameterized by ψ . The high-level actor observation is then formed by concatenating the local observation and graph embedding:

$$\mathbf{o}_t = \begin{bmatrix} \mathbf{o}_t^{\text{local}} \\ \mathbf{z}_t^{\text{GAT}} \end{bmatrix}. \quad (7)$$

During training, the high-level policy, parameterized by θ , samples

$$\mathbf{u}_t^\pi := [a_{f,t}^\pi, a_{l,t}^\pi, \omega_t^\pi]^\top \sim \pi_\theta(\cdot | \mathbf{o}_t), \quad (8)$$

where $a_{f,t}^\pi$ and $a_{l,t}^\pi$ are integrated over the high-level control interval $\Delta t > 0$ to obtain the forward and lateral velocity commands, while ω_t^π is directly used as the yaw-rate command. This action parameterization matches the reduced-order control input in (2), allowing DPCBF constraints to be evaluated directly on the high-level actor action. The resulting command $\mathbf{v}_{t+1}^{\text{cmd}} := [v_{f,t}^{\text{cmd}} + \Delta t a_{f,t}^\pi, v_{l,t}^{\text{cmd}} + \Delta t a_{l,t}^\pi, \omega_t^\pi]^\top$ is passed, together with proprioceptive observations, to the low-level policy, which generates the robot control commands $\mathbf{q}_t^{\text{des}}$.

C. Safety-Guided Reinforcement Learning

Unlike CBF-RL [13], DODGER executes the actor's output directly during training and uses the CBF-filtered reference and constraint violations only to guide learning.

1) *Safety Filter*: During training, DODGER computes a QP reference that penalizes deviations from the actor action $\mathbf{u}_t^\pi$ while incorporating DPCBF and input constraints. Its unrelaxed formulation is

$$\mathbf{u}_t^{\text{safe}} = \underset{\mathbf{u}_t}{\text{argmin}} \quad \frac{1}{2} \|\mathbf{u}_t - \mathbf{u}_t^\pi\|_2^2 \quad (9a)$$

$$\text{s.t.} \quad L_f h_j(\mathbf{x}_t) + L_g h_j(\mathbf{x}_t) \mathbf{u}_t + \alpha(h_j(\mathbf{x}_t)) \geq 0, \quad \forall j \in \{1, \dots, N_{\text{obs}}\}, \quad (9b)$$

$$v_f^{\text{lb}} \leq v_{f,t}^{\text{cmd}} + \Delta t a_{f,t}^\pi \leq v_f^{\text{ub}}, \quad (9c)$$

$$v_l^{\text{lb}} \leq v_{l,t}^{\text{cmd}} + \Delta t a_{l,t}^\pi \leq v_l^{\text{ub}}, \quad (9d)$$

$$\omega^{\text{lb}} \leq \omega_t \leq \omega^{\text{ub}}. \quad (9e)$$

Here, $\mathbf{u}_t = [a_{f,t}, a_{l,t}, \omega_t]^\top$ is the QP decision variable, and h_j denotes the DPCBF in (4) constructed for the j -th detected obstacle. The superscripts $(\cdot)^{\text{lb}}$ and $(\cdot)^{\text{ub}}$ denote the lower and upper command bounds. During training, the unrelaxed QP was feasible in 99.6% of solves; slack relaxation was used for the remaining 0.4%.

2) *Safety-Guided RL Training*: DODGER builds upon the Constraints as Terminations (CaT) principle [17], which maps soft constraint violations to termination probabilities used in return estimation. Rather than executing the CBF-filtered reference or terminating the rollout upon a constraint violation, DODGER continues the rollout with the actor action $\mathbf{u}_t^\pi$ and uses the resulting termination probability only when computing the learning target.

Let $\mathcal{I}_c$ and $\mathcal{I}_r$ denote the index sets of soft constraints and reward terms, respectively. We denote a constraint value by $c_{i_c,t} = c_{i_c}(\mathbf{x}_t, \mathbf{u}_t^\pi)$, where positive values indicate violations, and define $c_{i_c,t}^+ = \max(0, c_{i_c,t})$.

For the DPCBF constraint associated with the j -th obstacle, we define

$$c_{j,t}^{\text{DPCBF}} = -[L_f h_j(\mathbf{x}_t) + L_g h_j(\mathbf{x}_t) \mathbf{u}_t^\pi + \alpha(h_j(\mathbf{x}_t))]. \quad (10)$$

For the input constraints, let

$$\boldsymbol{\xi}_t^\pi = [v_{f,t}^{\text{cmd}} + \Delta t a_{f,t}^\pi, v_{l,t}^{\text{cmd}} + \Delta t a_{l,t}^\pi, \omega_t^\pi]^\top, \quad (11)$$

with corresponding lower and upper bounds $\boldsymbol{\xi}_t^{\text{lb}}$ and $\boldsymbol{\xi}_t^{\text{ub}}$. The violation of the k -th input constraint is defined as

$$c_{k,t}^{\text{input}} = \max\{\xi_k^{\text{lb}} - \xi_{k,t}^\pi, \xi_{k,t}^\pi - \xi_k^{\text{ub}}\}. \quad (12)$$

Algorithm 1 DODGER Training Procedure

```

1: Initialize  $\Theta$  and  $c_{i_c}^{\text{max}} > 0$  ( $i_c \in \mathcal{I}_c$ )
2: for step = 1 to  $N_{\text{steps}}$  do
3:   Initialize  $\mathbf{v}_0^{\text{cmd}}$ ,  $\mathbf{o}_0$ , and  $\mathbf{B} \leftarrow \emptyset$ 
4:   for  $t = 0$  to  $T - 1$  do
5:      $\mathbf{u}_t^\pi \sim \pi_\theta(\cdot | \mathbf{o}_t)$ 
6:      $\mathbf{u}_t^{\text{safe}} \leftarrow \text{SAFETYFILTER}(\mathbf{u}_t^\pi)$ 
7:      $(\mathbf{o}_{t+1}, d_t) \leftarrow \text{ENVIRONMENTUPDATE}(\mathbf{o}_t, \mathbf{u}_t^\pi)$ 
8:     for all  $i_c \in \mathcal{I}_c$  do
9:        $c_{i_c,t} \leftarrow c_{i_c}(\mathbf{x}_t, \mathbf{u}_t^\pi)$ 
10:       $c_{i_c,t}^+ \leftarrow \max\{0, c_{i_c,t}\}$ 
11:    end for
12:     $\delta_t \leftarrow \max_{i_c \in \mathcal{I}_c} \left[ p_{i_c}^{\text{max}} \text{clip} \left( \frac{c_{i_c,t}^+}{c_{i_c}^{\text{max}}}, 0, 1 \right) \right]$ 
13:     $r_t^{\text{CBF}} \leftarrow \exp \left( -\frac{\|\mathbf{u}_t^\pi - \mathbf{u}_t^{\text{safe}}\|_2^2}{\sigma^2} \right) - 1$ 
14:    Compute  $r_t^{i_r}$  ( $i_r \in \mathcal{I}_r$ ), including  $r_t^{\text{CBF}}$ 
15:     $R_t^+ \leftarrow 0$ ,  $R_t^- \leftarrow 0$ 
16:    for all  $i_r \in \mathcal{I}_r$  do
17:       $R_t^+ \leftarrow R_t^+ + \max(r_t^{i_r}, 0)$ 
18:       $R_t^- \leftarrow R_t^- + \min(r_t^{i_r}, 0)$ 
19:    end for
20:    Store  $(\mathbf{o}_t, \mathbf{u}_t^\pi, \delta_t, R_t^+, R_t^-, d_t, \mathbf{o}_{t+1})$  in  $\mathbf{B}$ 
21:  end for
22:  for all  $i_c \in \mathcal{I}_c$  do
23:     $c_{i_c}^{\text{max}} \leftarrow \tau_c c_{i_c}^{\text{max}} + (1 - \tau_c) \max_{0 \leq t < T} c_{i_c,t}^+$ 
24:  end for
25:   $\Theta \leftarrow \text{UPDATEPPO}(\Theta, \mathbf{B})$ 
26: end for

```

Following CaT, each constraint $c_{i_c,t}$ is associated with a maximum termination probability $p_{i_c}^{\text{max}}$ that controls its enforcement strength. The termination probability is computed from the normalized constraint violations as

$$\delta_t = \max_{i_c \in \mathcal{I}_c} \left[p_{i_c}^{\text{max}} \text{clip} \left(\frac{c_{i_c,t}^+}{c_{i_c}^{\text{max}}}, 0, 1 \right) \right]. \quad (13)$$

Larger violations increase δ_t , reducing the positive return and bootstrapping contributions while the rollout continues with the actor action $\mathbf{u}_t^\pi$.

The normalization scale $c_{i_c}^{\text{max}}$ is updated from the maximum constraint violation observed during each rollout using

$$c_{i_c}^{\text{max}} \leftarrow \tau_c c_{i_c}^{\text{max}} + (1 - \tau_c) \max_{0 \leq t < T} c_{i_c,t}^+, \quad (14)$$

where $\tau_c \in (0, 1)$ is the exponential smoothing coefficient.

DODGER additionally uses the safety-filtered action as a reference through

$$r_t^{\text{CBF}} = \exp \left(-\frac{\|\mathbf{u}_t^\pi - \mathbf{u}_t^{\text{safe}}\|_2^2}{\sigma^2} \right) - 1, \quad (15)$$

which penalizes deviations of the actor action from the QP reference.

Directly applying the CaT factor $(1 - \delta_t)$ to signed rewards can undesirably weaken negative penalties, since multiplication by $(1 - \delta_t)$ reduces their magnitude. This is particularly relevant to navigation tasks, where a linear progress reward naturally becomes negative when the robot moves away from the goal, while terminal events such as collisions and workspace violations require penalties with different severities. Transforming all rewards into positive quantities can instead distort their intended relative importance.

To prevent the CaT factor from weakening task-specific negative penalties, DODGER separates the reward terms into positive and negative components and employs a two-head critic. The positive and negative reward sums are denoted by R_t^+ and R_t^- , respectively. Inspired by the multi-head value architecture in [18], the critic independently estimates V^+ and V^- . The factor $(1 - \delta_t)$ is applied only to the positive branch, while the negative branch preserves the original penalty magnitudes.

The positive-branch temporal-difference (TD) error is

$$\epsilon_t^+ = (1 - \delta_t)R_t^+ + \gamma(1 - d_t)(1 - \delta_t)V^+(\mathbf{o}_{t+1}) - V^+(\mathbf{o}_t),$$

where $\gamma \in [0, 1]$ is the discount factor and $d_t \in \{0, 1\}$ indicates environment termination. The same factor $(1 - \delta_t)$ is applied to the positive-branch return estimation, whereas the negative branch follows the standard proximal policy optimization (PPO) update [19]. The critic loss is $L_V = \frac{1}{2}L_V^+ + \frac{1}{2}L_V^-$, where L_V^+ and L_V^- are the corresponding value losses. The two advantages are combined and normalized for the PPO update as $A_t = \text{Normalize}(A_t^+ + A_t^-)$.

Thus, DODGER incorporates safety guidance through two complementary learning signals. The action-correction reward r_t^{CBF} penalizes deviations from the CBF-filtered reference and, because $r_t^{\text{CBF}} \in (-1, 0]$, contributes to the negative reward branch. In parallel, the constraint-dependent factor δ_t reduces the positive branch according to the severity of constraint violations. Neither mechanism replaces the actor action during environment interaction, thereby preserving policy-generated rollouts. Algorithm 1 summarizes training, with trainable parameters Θ , rollout buffer $\mathbf{B}$, rollout length T , and N_{steps} policy updates.

IV. RESULTS

A. Low-Dimensional Validation

We first evaluate the safety representation learned by DODGER using a low-dimensional Dubins-car benchmark following [20]. Since classical grid-based HJ reachability scales exponentially with state dimension, direct online application to our five-dimensional reduced-order navigation model is computationally impractical [21]; the three-dimensional benchmark therefore provides a tractable HJ reference. The state is $\mathbf{x} = [x, y, \phi]^\top$ with dynamics $\dot{\mathbf{x}} = [v \cos \phi, v \sin \phi, 0]^\top + [0, 0, 1]^\top \omega$, where $v = 1 \text{ m/s}$ and $\omega \in [-1.25, 1.25] \text{ rad/s}$.

The obstacle is a circle centered at the origin with radius $r_{\text{obs}} = 0.5 \text{ m}$, defining the occupied state region $\mathcal{O} = \{\mathbf{x} \in \mathcal{X} \mid x^2 + y^2 \leq r_{\text{obs}}^2\}$. Given the known dynamics, we compute a DPCBF-based HJ reference by backward reachability from $\{h \leq 0\}$ [21], using $V_{\text{HJ}}(\mathbf{x}, T) = h(\mathbf{x})$. Here, T is the reachability horizon, and $V_{\text{HJ}}(\mathbf{x}) = V_{\text{HJ}}(\mathbf{x}, 0)$. We define the HJ safety boundary as $\partial V_{\text{HJ}} = \{\mathbf{x} \in \mathcal{X} \mid V_{\text{HJ}}(\mathbf{x}) = 0\}$. Using $V_{\text{HJ}}(\mathbf{x}) \leq 0$ for unsafe states, the HJ reference unsafe set is $\mathcal{U}_{\text{HJ}} = \{\mathbf{x} \in \mathcal{X} \mid V_{\text{HJ}}(\mathbf{x}) \leq 0\}$.

For this diagnostic, both methods use negative-reward critics with identical reward scaling, evaluated on the benchmark state $\mathbf{x}$. Their critic-induced risk sets are

$$\hat{\mathcal{U}}_\epsilon = \mathcal{O} \cup \{\mathbf{x} \in \mathcal{X} \setminus \mathcal{O} \mid V^-(\mathbf{x}) \leq \epsilon\},$$

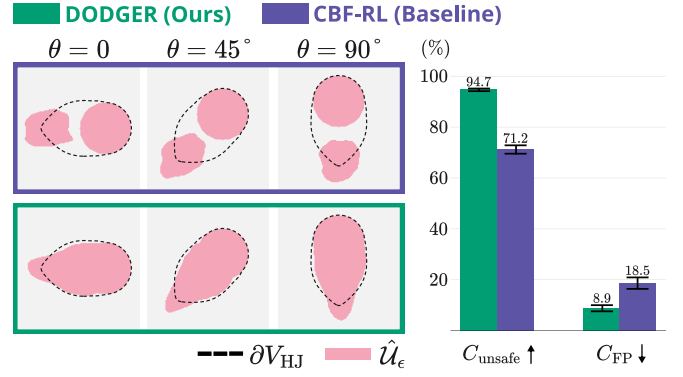


Fig. 3: Comparison of DODGER and CBF-RL on the Dubins-car benchmark. The dashed black curve denotes the HJ safety boundary ∂V_{HJ} , and the shaded region denotes $\hat{\mathcal{U}}_\epsilon$.

where the same threshold $\epsilon = -0.5$ is used for all methods.

We quantify its agreement with the HJ reference using

$$C_{\text{unsafe}} = \frac{\mu(\hat{\mathcal{U}}_\epsilon \cap \mathcal{U}_{\text{HJ}})}{\mu(\mathcal{U}_{\text{HJ}})}, \quad C_{\text{FP}} = \frac{\mu(\hat{\mathcal{U}}_\epsilon \setminus \mathcal{U}_{\text{HJ}})}{\mu(\mathcal{U}_{\text{HJ}})}.$$

Here, μ denotes set measure, C_{unsafe} is unsafe-set coverage, and C_{FP} is the false-positive measure normalized by $\mu(\mathcal{U}_{\text{HJ}})$.

The reported metrics are averaged over five random seeds and are given as mean $\pm$ standard deviation. As shown in Fig. 3, DODGER achieves $C_{\text{unsafe}} = 94.7 \pm 0.93\%$ and $C_{\text{FP}} = 8.9 \pm 2.47\%$, compared with $71.2 \pm 3.47\%$ and $18.5 \pm 4.62\%$ for CBF-RL, respectively. These results indicate that the critic-induced risk region learned by DODGER more closely matches the HJ reference set, with higher coverage and fewer false positives.

B. Humanoid Dynamic-Obstacle Avoidance

1) *Experimental Setup*: We evaluate DODGER on full-order humanoid navigation among multiple dynamic obstacles. The high-level navigation policy is trained in MuJoCo Playground [22] at 10 Hz, while a pretrained Unitree G1 velocity-tracking policy operates at 50 Hz. We additionally provide an interactive web demo using MuJoCo compiled to WebAssembly.³

Training uses a four-stage curriculum that progressively increases obstacle density and velocity, state randomization, and observation noise, with the final stage containing 20 dynamic obstacles at speeds up to 0.6 m/s. At each high-level step, up to 10 obstacles within the 5 m sensing range are retained for GAT encoding and the multi-constraint QP, prioritized by the alignment of relative velocity toward the robot and by distance; unused slots are padded and masked. The policy is trained for up to 600 million environment transitions and, after reaching the final curriculum stage, evaluated every 3 million transitions over 256 episodes. An episode is successful if the robot reaches the goal within 0.6 m before failure or timeout while satisfying $|\text{wrap}(\phi_g - \phi_t)| \leq 10^\circ$, $\|\mathbf{v}_t^b\|_2 \leq 0.2 \text{ m/s}$, and $|\dot{\phi}_t| \leq 0.2 \text{ rad/s}$. Convergence is defined as a mean success rate of at least 95% over the three most recent evaluations.

³Web demo: <https://dodger.taekyung.me>

TABLE I: Reward terms used for training DODGER.

Term	Definition	Weight
r_{CBF}	$\exp(-\ \Delta \mathbf{u}\ _2^2 / \sigma^2) - 1$	100
r_{prog}	$\Delta \rho$	60
r_{head}	$\Delta \bar{e}_\phi \exp[-(\rho/\ell_h)^2]$	10
r_{goal}	$\mathbb{1}\{\text{goal reached}\}$	20
r_{obs}	$\mathbb{1}\{\text{obstacle collision}\}$	-55
r_{outside}	$\mathbb{1}\{\mathbf{p} \notin \mathcal{A}_{\text{hard}}\}$	-100
r_{time}	1	-0.01
r_{timeout}	$\mathbb{1}\{\text{timeout}\}$	-15
r_{rate}	$\ \Delta \tilde{\mathbf{u}}^\pi\ _2^2$	-0.02

The command bounds are $v_f^{\text{cmd}} \in [-1, 2] \text{ m/s}$, $v_l^{\text{cmd}} \in [-1, 1] \text{ m/s}$, and $\omega^{\text{cmd}} \in [-1, 1] \text{ rad/s}$, while the actor accelerations are clipped to $a_f^\pi \in [-4, 4] \text{ m/s}^2$ and $a_l^\pi \in [-2, 2] \text{ m/s}^2$. For the CaT update, DPCBF and input-constraint violations are treated as soft constraints, with $p_{i_c}^{\text{max}}$ gradually increased to 0.25 [17].

At each of the five low-level steps, we recompute the QP reference and constraint values. The CBF reward is averaged and the constraint violation is max-pooled over the interval: $r_t^{\text{CBF}} = \frac{1}{5} \sum_{\ell=1}^5 r_{t,\ell}^{\text{CBF}}$ and $c_{i_c,t}^+ = \max_{\ell \in \{1,\dots,5\}} c_{i_c,t,\ell}^+$.

The reward terms and their weights are summarized in Table I. Let $\mathbf{p}_t$ and $\mathbf{p}_g$ denote the world-frame robot and goal positions, and define $\rho_t = \|\mathbf{p}_t - \mathbf{p}_g\|_2$, $\Delta \rho_t = \rho_t - \rho_{t+1}$, $\bar{e}_{\phi,t} = |\text{wrap}(\phi_g - \phi_t)|$, $\Delta \bar{e}_{\phi,t} = \bar{e}_{\phi,t} - \bar{e}_{\phi,t+1}$, $\Delta \mathbf{u}_t = \mathbf{u}_t^\pi - \mathbf{u}_t^{\text{safe}}$, and $\Delta \tilde{\mathbf{u}}_t^\pi = \tilde{\mathbf{u}}_t^\pi - \tilde{\mathbf{u}}_{t-1}^\pi$. Here, $\tilde{\mathbf{u}}_t^\pi$ denotes the componentwise normalized actor action. We use $\sigma = 0.5$ and $\ell_h = 1.0 \text{ m}$. Here, $\mathcal{A}_{\text{hard}}$ denotes the prescribed workspace. Each reward term is multiplied by its listed weight before being separated into positive and negative components as described in Sec. III-C.2.

Dynamic obstacles are detected and tracked within a 5m radius using LiDAR and the ROS 2 `obstacle_detector_2` package [23].

2) *RL Training Comparison:* We compare DODGER with two CBF-RL baselines under the same training configuration to evaluate learning efficiency in dense dynamic-obstacle navigation. Our CBF-RL baseline, adapted from [13], executes safety-filtered actions during training and sums the CBF-condition penalties over the detected obstacles. Like CBF-RL, CBF-RL-Min executes the safety-filtered action during training, but uses only the most critical obstacle-wise term, $\min_j \text{clip}(\dot{h}_j + \alpha(h_j), -c, 0)$, with clipping bound $c > 0$, to avoid accumulating CBF penalties across multiple obstacles, following [24].

As shown in Fig. 4, DODGER converges after approximately 114 million environment transitions, whereas CBF-RL-Min requires approximately 277 million. In contrast, CBF-RL does not reach the convergence criterion within 600 million transitions and achieves a final success rate of 72.3% beyond the plotted range.

We further evaluate the methods at obstacle speeds up to 0.8 m/s with a 1200-million-transition budget. DODGER converges after approximately 536 million transitions, whereas CBF-RL-Min and CBF-RL do not converge, with final success rates of 88.5% and 0.4%, respectively.

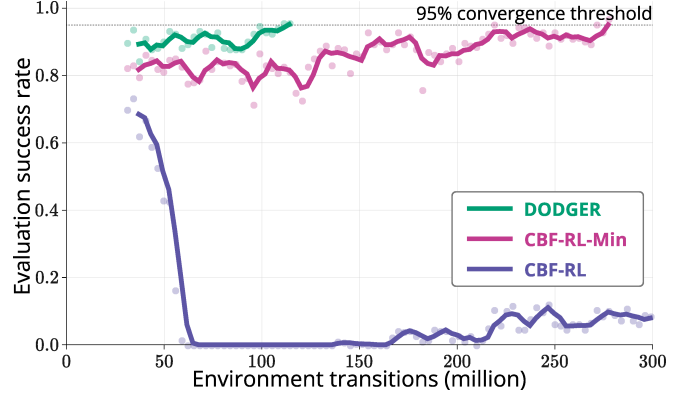


Fig. 4: Evaluation success rates of DODGER, CBF-RL-Min, and CBF-RL during the first 300 million environment transitions for humanoid navigation with obstacle speeds up to 0.6 m/s. The markers denote individual evaluations, and the solid lines denote the moving average over the three most recent evaluations.

TABLE II: Training outcomes under different CBF formulations. “End” denotes the number of environment transitions (millions) at which training terminated; curriculum stages are indexed from 0 to 3.

CBF formulation	End (M)	Converged	Stage
DPCBF (DODGER)	113.97	✓	3
C3BF	600	✗	1
Dist-ECBF	166.20	✓	3
Dist-ECBF + yaw	154.01	✓	3

3) *CBF Formulation Comparison:* We compare DPCBF with the Collision Cone CBF (C3BF) [25] and a distance-based CBF under the same DODGER training configuration. C3BF incorporates relative position and velocity through $h_{\text{C3}} = \langle \mathbf{p}_{\text{rel}}, \mathbf{v}_{\text{rel}} \rangle + d(\mathbf{x}) \|\mathbf{v}_{\text{rel}}\|$. In contrast, the distance-based barrier is defined as $h_{\text{dist}} = \|\mathbf{p}_{\text{rel}}\|^2 - r^2$. Since h_{dist} has relative degree two with respect to the reduced-order dynamics in (2), we enforce the distance-based safety constraint using an exponential CBF (ECBF) [26]. We additionally evaluate a distance-based ECBF variant with yaw-rate regularization to examine the large yaw commands observed with the original distance-based formulation. All other components are kept identical across the CBF variants.

Table II summarizes the training outcomes. DPCBF and both distance-based ECBF variants reach curriculum stage 3 and converge, with DPCBF requiring the fewest environment transitions. In contrast, C3BF does not converge within the training budget and reaches only stage 1.

To further characterize the yaw behavior, we compare the actor yaw commands and the resulting G1 yaw rates for DPCBF and the two distance-based ECBF variants. Over N evaluated samples, the command root-mean-square (RMS) value is $\omega_{\text{cmd,RMS}} = \sqrt{N^{-1} \sum_{t=1}^N (\omega_t^\pi)^2}$, while the command near-saturation fraction is $f_{\text{cmd,sat}} = N^{-1} \sum_{t=1}^N \mathbb{1}\{|\omega_t^\pi| \geq 0.9 \text{ rad/s}\}$. The G1 metrics use the measured yaw rate $\dot{\phi}_t$ and the same threshold; fractions are reported as percentages. As shown in Table III, the distance-based ECBF variants produce larger yaw-rate commands and more frequent near-saturation than DPCBF, both in the actor output and in the resulting G1 motion.

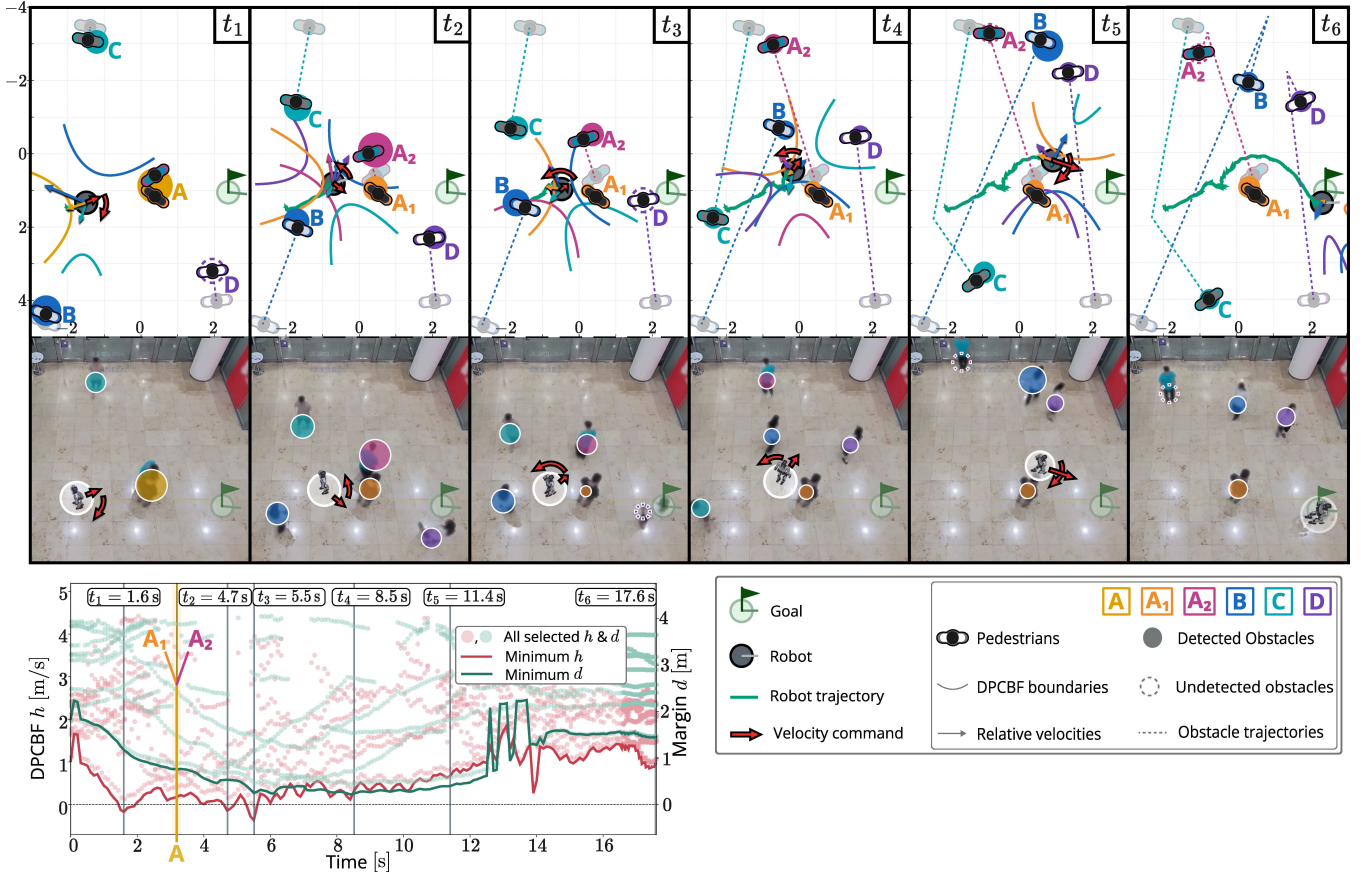


Fig. 5: Real-robot navigation with a Unitree G1 among multiple moving pedestrians. The upper panels show the estimated scene and camera views at six representative times, together with DPCBF boundaries and relative velocities; $h < 0$ when a relative velocity enters the unsafe side of its corresponding parabolic boundary. The lower-left plot shows the DPCBF values h and margins d of the selected obstacles, with solid curves denoting their minima. Although the minimum h temporarily falls below zero, the minimum d remains positive throughout the experiment.

TABLE III: Yaw-rate statistics for different CBF formulations.

Metric	DPCBF	Dist.	Dist.+Yaw
Cmd. RMS (rad/s)	0.554	0.934	0.894
Cmd. near-sat. (%)	5.48	87.08	61.56
G1 RMS (rad/s)	0.435	0.808	0.763
G1 near-sat. (%)	0.87	18.80	15.45

C. Discussion

The graph-based encoder accommodates more than the training limit of 10 observed obstacles. Removing this cap at evaluation yields a 97.27% success rate, with 12.54 observed obstacles on average and up to 20 simultaneously; 68.65% of decisions involve more than 10 obstacles.

Across the low-dimensional HJ benchmark and full-order humanoid navigation, the results support preserving unfiltered policy interaction: DODGER more closely recovers the HJ reference region and converges faster than the safety-filtered CBF-RL baselines.

The CBF comparison highlights the importance of the safety representation. C3BF achieves only a 23% final success rate, with 76% of episodes terminating by timeout and only 1% by collision, indicating overly conservative behavior in dense scenes. The distance-based ECBF converges but

produces excessive rotational motion. For $h_{\text{dist}} = \|\mathbf{p}_{\text{rel}}\|^2 - r^2$, the yaw coefficient in the higher-order CBF condition is

$$L_g L_f h_{\text{dist}}|_{\omega} = 2(p_{\text{rel},y}\dot{x} - p_{\text{rel},x}\dot{y}), \quad (16)$$

which vanishes when the robot velocity is collinear with $\mathbf{p}_{\text{rel}}$, making the instantaneous ECBF constraint insensitive to yaw in these configurations. Yaw regularization reduces this behavior but does not eliminate it. In contrast, DPCBF yields substantially lower yaw-rate RMS and near-saturation, suggesting that its LoS-based relative-motion representation provides more effective directional guidance.

D. Real-Robot Experiments

We deploy the DODGER policy trained with obstacle speeds up to 0.8 m/s on a Unitree G1 among multiple moving pedestrians. The floating-base pose is estimated from LiDAR and IMU measurements using Direct LiDAR-Inertial Odometry (DLIO) [27], and at most 10 detected obstacles are selected per step for GAT encoding and DPCBF evaluation.

As shown in Fig. 5, five pedestrians, denoted by A_1 , A_2 , B , C , and D , move around the robot as it navigates toward the goal. Real-world perception introduces merge-split errors and abrupt changes in estimated obstacle geometry and velocity. For example, A_1 and A_2 are initially perceived

as a single obstacle A ; although they physically separate at approximately 3.18s, the detector resolves them as A_1 and A_2 only later. Imperfect locomotion tracking also causes deviations between commanded and realized robot motion.

The lower-left plot in Fig. 5 shows h and d for the selected obstacles. The minimum h is negative around t_1 , t_2 , and t_3 , whereas the minimum d remains positive, indicating separation of the estimated footprints despite temporary violations of $h \geq 0$. These negative values are associated with abrupt changes in perceived obstacle states, deviations from the constant-velocity obstacle model, and realized robot motion. Around t_1 , pedestrians A_1 and A_2 are still represented as a single nearly stationary obstacle A ; changes in its estimated radius induce a spurious nonzero velocity, causing $h < 0$. Around t_2 , pedestrian B is estimated at $v_B = 0.895$ m/s, exceeding the 0.8 m/s training range. After A is split into A_1 and A_2 , a similar geometry-induced velocity error occurs around t_3 , where the nearly stationary A_1 is temporarily perceived as approaching the robot, producing the minimum observed DPCBF value while the distance-dependent margin remains positive ($d > 0$). Despite these transient negative DPCBF values and perception errors, the robot avoids collision and reaches the goal.

V. CONCLUSION

In this paper, we introduced DODGER, which combines direct policy interaction with DPCBF-based safety guidance to learn collision-avoidance behavior without requiring a runtime safety filter. Low-dimensional validation showed close agreement with the HJ reference, while full-order humanoid simulations demonstrated efficient learning in dense multi-obstacle navigation and the effectiveness of DPCBF over alternative CBF formulations. Real-world humanoid experiments further showed successful navigation among multiple moving obstacles under perception and execution uncertainties. Future work will extend the framework toward richer three-dimensional obstacle representations and full-order, perception-aware safety modeling for more reliable navigation in complex human environments.

REFERENCES

- [1] E. Kaufmann, L. Bauersfeld, A. Loquercio, M. Müller, V. Koltun, and D. Scaramuzza, "Champion-level drone racing using deep reinforcement learning," *Nature*, vol. 620, pp. 982–987, 2023.
- [2] I. Radosavovic, T. Xiao, B. Zhang, T. Darrell, J. Malik, and K. Sreenath, "Real-world humanoid locomotion with reinforcement learning," *Science Robotics*, vol. 9, no. 89, p. eadi9579, 2024.
- [3] O. Andersson, M. Wzorek, P. Rudol, and P. Doherty, "Model-predictive control with stochastic collision avoidance using bayesian policy optimization," in *IEEE International Conference on Robotics and Automation (ICRA)*, 2016, pp. 4597–4604.
- [4] M. Gaertner, M. Bjelonic, F. Farshidian, and M. Hutter, "Collision-Free MPC for legged robots in static and dynamic scenes," in *IEEE International Conference on Robotics and Automation (ICRA)*, 2021, pp. 8266–8272.
- [5] M. Harms, M. Jacquet, and K. Alexis, "Safe quadrotor navigation using composite control barrier functions," in *IEEE International Conference on Robotics and Automation (ICRA)*, 2025, pp. 6343–6349.
- [6] L. Yang, B. Werner, R. K. Cosner, D. Fridovich-Keil, P. Culbertson, and A. D. Ames, "SHIELD: Safety on humanoids via CBFs in expectation on learned dynamics," in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2025, pp. 203–210.
- [7] T. Kim, "Is Your Safe Controller Actually Safe? A Critical Review of CBF Tautologies and Hidden Assumptions," in *arXiv preprint arXiv:2603.06954*, 2026.
- [8] N. Nilaksh, A. Ranjan, S. Agrawal, A. Jain, P. Jagtap, and S. Kolathaya, "Barrier functions inspired reward shaping for reinforcement learning," in *IEEE International Conference on Robotics and Automation (ICRA)*, 2024, pp. 10807–10813.
- [9] X. Wang, T. Kim, B. Hoxha, G. Fainekos, and D. Panagou, "Reinforcement learning for risk adaptation via differentiable CVaR barrier functions," *arXiv preprint arXiv:2605.21257*, 2026.
- [10] R. Cheng, G. Orosz, R. M. Murray, and J. W. Burdick, "End-to-end safe reinforcement learning through barrier functions for safety-critical continuous control tasks," in *Proceedings of the AAAI Conference on Artificial Intelligence*, 2019, pp. 3387–3395.
- [11] Y. Emam, G. Notomista, P. Giotfelter, Z. Kira, and M. Egerstedt, "Safe reinforcement learning using robust control barrier functions," *IEEE Robotics and Automation Letters*, vol. 10, no. 3, pp. 2886–2893, 2025.
- [12] F. Pizarro Bejarano, L. Brunke, and A. P. Schoellig, "Safety filtering while training: Improving the performance and sample efficiency of reinforcement learning agents," *IEEE Robotics and Automation Letters*, vol. 10, no. 1, pp. 788–795, 2025.
- [13] L. Yang, B. Werner, M. de Sa, and A. D. Ames, "CBF-RL: Safety filtering reinforcement learning in training with control barrier functions," in *IEEE International Conference on Robotics and Automation (ICRA)*, 2026.
- [14] H. K. Park, T. Kim, and D. Panagou, "Beyond collision cones: Dynamic obstacle avoidance for nonholonomic robots via dynamic parabolic control barrier functions," in *IEEE International Conference on Robotics and Automation (ICRA)*, 2026.
- [15] A. D. Ames, S. Coogan, M. Egerstedt, G. Notomista, K. Sreenath, and P. Tabuada, "Control barrier functions: Theory and applications," in *European Control Conference (ECC)*, 2019, pp. 3420–3431.
- [16] T. Kim, R. I. Kee, and D. Panagou, "Learning to adapt control barrier functions under epistemic and aleatoric uncertainty," in *arXiv preprint arXiv:2504.03038*, 2026.
- [17] E. Chane-Sane, P.-A. Leziart, T. Flayols, O. Stasse, P. Souères, and N. Mansard, "CaT: Constraints as terminations for legged locomotion reinforcement learning," in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2024, pp. 13 303–13 310.
- [18] H. Munn, B. Tidd, P. Böhm, M. Gallagher, and D. Howard, "Scalable multi-objective robot reinforcement learning through gradient conflict resolution," in *IEEE International Conference on Robotics and Automation (ICRA)*, 2026.
- [19] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," in *arXiv preprint arXiv:1707.06347*, 2017.
- [20] J. Seo, K. Nakamura, and A. Bajcsy, "Uncertainty-aware latent safety filters for avoiding out-of-distribution failures," in *Conference on Robot Learning (CoRL)*, 2025.
- [21] I. M. Mitchell, A. M. Bayen, and C. J. Tomlin, "A time-dependent Hamilton–Jacobi formulation of reachable sets for continuous dynamic games," *IEEE Transactions on Automatic Control*, vol. 50, no. 7, pp. 947–957, 2005.
- [22] K. Zakka, B. Tabanpour, Q. Liao, M. Haiderbhai, S. Holt, J. Y. Luo, A. Allshire, E. Frey, K. Sreenath, L. A. Kahrs, C. Sferazza, Y. Tassa, and P. Abbeel, "MuJoCo Playground," in *arXiv preprint arXiv:2502.08844*, 2025.
- [23] M. Przybyła, "Detection and tracking of 2D geometric obstacles from LRF data," in *International Workshop on Robot Motion and Control (RoMoCo)*, 2017, pp. 135–141.
- [24] L. Yang, J. Li, and A. D. Ames, "PAC-MAN: Perception-aware CBF-RL for whole-body safety in humanoid dodgeball," in *arXiv preprint arXiv:2607.28623*, 2026.
- [25] M. Tayal, B. Giri Goswami, K. Rajgopal, R. Singh, M. Tejas Rao, J. Keshavan, P. Jagtap, and S. Nadubettu Yadukumar, "A collision cone approach for control barrier functions," *IEEE Transactions on Control Systems Technology*, vol. 34, no. 3, pp. 1297–1310, 2026.
- [26] Q. Nguyen and K. Sreenath, "Exponential control barrier functions for enforcing high relative-degree safety-critical constraints," in *American Control Conference (ACC)*, 2016, pp. 322–328.
- [27] K. Chen, R. Nemiroff, and B. T. Lopez, "Direct LiDAR-inertial odometry: Lightweight LIO with continuous-time motion correction," in *IEEE International Conference on Robotics and Automation (ICRA)*, 2023, pp. 3983–3989.

APPENDIX

A. Barrier Function Derivatives and Safety Conditions

1) *Common Notation:* The obstacle index j is omitted throughout this appendix for clarity. Recall that

$$\mathbf{p}_{\text{rel}} = \mathbf{p}_{\text{rob}} - \mathbf{p}_{\text{obs}}, \quad \mathbf{v}_{\text{rel}} = \mathbf{v}_{\text{rob}} - \mathbf{v}_{\text{obs}},$$

and

$$d(\mathbf{x}) = \sqrt{\|\mathbf{p}_{\text{rel}}\|^2 - r^2}.$$

Let

$$\beta = \text{atan2}(p_{\text{rel},y}, p_{\text{rel},x})$$

denote the line-of-sight (LoS) angle and define $\tilde{\phi} = \phi - \beta$. The relative velocity expressed in the LoS frame is

$$\tilde{v}_{\text{rel},x} = v_{\text{rel},x} \cos \beta + v_{\text{rel},y} \sin \beta, \quad (17)$$

$$\tilde{v}_{\text{rel},y} = -v_{\text{rel},x} \sin \beta + v_{\text{rel},y} \cos \beta. \quad (18)$$

The DPCBF is

$$h_{\text{DPCBF}} = \tilde{v}_{\text{rel},x} + k_\lambda \frac{d(\mathbf{x})}{\|\mathbf{v}_{\text{rel}}\|} \tilde{v}_{\text{rel},y}^2 + k_\mu d(\mathbf{x}). \quad (19)$$

2) *DPCBF for the Reduced-Order Model:* For the reduced-order dynamics in (2), the drift Lie derivative of (19) is

$$\begin{aligned} L_f h_{\text{DPCBF}} &= \frac{\tilde{v}_{\text{rel},y}^2}{\|\mathbf{p}_{\text{rel}}\|} \\ &+ k_\lambda \frac{\tilde{v}_{\text{rel},x} \tilde{v}_{\text{rel},y}^2}{\|\mathbf{v}_{\text{rel}}\|} \left(\frac{\|\mathbf{p}_{\text{rel}}\|}{d(\mathbf{x})} - \frac{2d(\mathbf{x})}{\|\mathbf{p}_{\text{rel}}\|} \right) \\ &+ k_\mu \frac{\|\mathbf{p}_{\text{rel}}\|}{d(\mathbf{x})} \tilde{v}_{\text{rel},x}. \end{aligned} \quad (20)$$

The input Lie derivative is written as

$$L_g h_{\text{DPCBF}} = [\ell_f(\mathbf{x}) \quad \ell_l(\mathbf{x}) \quad \ell_\omega(\mathbf{x})], \quad (21)$$

where

$$\begin{aligned} \ell_f(\mathbf{x}) &= \left(1 - k_\lambda \frac{d(\mathbf{x})}{\|\mathbf{v}_{\text{rel}}\|^3} \tilde{v}_{\text{rel},x} \tilde{v}_{\text{rel},y}^2 \right) \cos \tilde{\phi} \\ &+ k_\lambda d(\mathbf{x}) \left(\frac{2\tilde{v}_{\text{rel},y}}{\|\mathbf{v}_{\text{rel}}\|} - \frac{\tilde{v}_{\text{rel},y}^3}{\|\mathbf{v}_{\text{rel}}\|^3} \right) \sin \tilde{\phi}, \end{aligned} \quad (22)$$

$$\begin{aligned} \ell_l(\mathbf{x}) &= - \left(1 - k_\lambda \frac{d(\mathbf{x})}{\|\mathbf{v}_{\text{rel}}\|^3} \tilde{v}_{\text{rel},x} \tilde{v}_{\text{rel},y}^2 \right) \sin \tilde{\phi} \\ &+ k_\lambda d(\mathbf{x}) \left(\frac{2\tilde{v}_{\text{rel},y}}{\|\mathbf{v}_{\text{rel}}\|} - \frac{\tilde{v}_{\text{rel},y}^3}{\|\mathbf{v}_{\text{rel}}\|^3} \right) \cos \tilde{\phi}, \end{aligned} \quad (23)$$

$$\begin{aligned} \ell_\omega(\mathbf{x}) &= - \left(1 - k_\lambda \frac{d(\mathbf{x})}{\|\mathbf{v}_{\text{rel}}\|^3} \tilde{v}_{\text{rel},x} \tilde{v}_{\text{rel},y}^2 \right) (v_f \sin \tilde{\phi} + v_l \cos \tilde{\phi}) \\ &+ k_\lambda d(\mathbf{x}) \left(\frac{2\tilde{v}_{\text{rel},y}}{\|\mathbf{v}_{\text{rel}}\|} - \frac{\tilde{v}_{\text{rel},y}^3}{\|\mathbf{v}_{\text{rel}}\|^3} \right) (v_f \cos \tilde{\phi} - v_l \sin \tilde{\phi}). \end{aligned} \quad (24)$$

Accordingly, the DPCBF condition used for the reduced-order model is

$$L_f h_{\text{DPCBF}} + L_g h_{\text{DPCBF}} [a_f \quad a_l \quad \omega]^\top + \alpha (h_{\text{DPCBF}}) \geq 0. \quad (25)$$

3) *DPCBF for the Dubins-Car Model*: For the Dubins-car benchmark, the state and control are $\mathbf{x} = [x, y, \phi]^\top$ and ω , respectively, with constant forward speed v . The obstacle is stationary, such that

$$\mathbf{v}_{\text{rel}} = [v \cos \phi \quad v \sin \phi]^\top.$$

We denote the corresponding barrier by $h_{\text{DPCBF}}^{\text{Dubins}}$.

Its drift Lie derivative is

$$\begin{aligned} L_f h_{\text{DPCBF}}^{\text{Dubins}} &= \frac{\tilde{v}_{\text{rel},y}^2}{\|\mathbf{p}_{\text{rel}}\|} \\ &+ k_\lambda \frac{\tilde{v}_{\text{rel},x} \tilde{v}_{\text{rel},y}^2}{\|\mathbf{v}_{\text{rel}}\|} \left(\frac{\|\mathbf{p}_{\text{rel}}\|}{d(\mathbf{x})} - \frac{2d(\mathbf{x})}{\|\mathbf{p}_{\text{rel}}\|} \right) \\ &+ k_\mu \frac{\|\mathbf{p}_{\text{rel}}\|}{d(\mathbf{x})} \tilde{v}_{\text{rel},x}, \end{aligned} \quad (26)$$

and its input Lie derivative is

$$L_g h_{\text{DPCBF}}^{\text{Dubins}} = -\tilde{v}_{\text{rel},y} + 2k_\lambda \frac{d(\mathbf{x})}{\|\mathbf{v}_{\text{rel}}\|} \tilde{v}_{\text{rel},x} \tilde{v}_{\text{rel},y}. \quad (27)$$

The corresponding first-order CBF condition is

$$L_f h_{\text{DPCBF}}^{\text{Dubins}} + L_g h_{\text{DPCBF}}^{\text{Dubins}} \omega + \alpha (h_{\text{DPCBF}}^{\text{Dubins}}) \geq 0. \quad (28)$$

4) *Collision Cone CBF*: The Collision Cone CBF (C3BF) used in the comparison is

$$h_{\text{C3}} = \mathbf{p}_{\text{rel}}^\top \mathbf{v}_{\text{rel}} + d(\mathbf{x}) \|\mathbf{v}_{\text{rel}}\|. \quad (29)$$

Its drift Lie derivative is

$$L_f h_{\text{C3}} = \|\mathbf{v}_{\text{rel}}\|^2 + \frac{\|\mathbf{v}_{\text{rel}}\|}{d(\mathbf{x})} \mathbf{p}_{\text{rel}}^\top \mathbf{v}_{\text{rel}}. \quad (30)$$

The input Lie derivative is

$$L_g h_{\text{C3}} = [\ell_f^{\text{C3}} \quad \ell_l^{\text{C3}} \quad \ell_\omega^{\text{C3}}], \quad (31)$$

where

$$\begin{aligned} \ell_f^{\text{C3}} &= \left(p_{\text{rel},x} + \frac{d(\mathbf{x})}{\|\mathbf{v}_{\text{rel}}\|} v_{\text{rel},x} \right) \cos \phi \\ &+ \left(p_{\text{rel},y} + \frac{d(\mathbf{x})}{\|\mathbf{v}_{\text{rel}}\|} v_{\text{rel},y} \right) \sin \phi, \end{aligned} \quad (32)$$

$$\begin{aligned} \ell_l^{\text{C3}} &= - \left(p_{\text{rel},x} + \frac{d(\mathbf{x})}{\|\mathbf{v}_{\text{rel}}\|} v_{\text{rel},x} \right) \sin \phi \\ &+ \left(p_{\text{rel},y} + \frac{d(\mathbf{x})}{\|\mathbf{v}_{\text{rel}}\|} v_{\text{rel},y} \right) \cos \phi, \end{aligned} \quad (33)$$

$$\begin{aligned} \ell_\omega^{\text{C3}} &= - \left(p_{\text{rel},x} + \frac{d(\mathbf{x})}{\|\mathbf{v}_{\text{rel}}\|} v_{\text{rel},x} \right) \dot{y} \\ &+ \left(p_{\text{rel},y} + \frac{d(\mathbf{x})}{\|\mathbf{v}_{\text{rel}}\|} v_{\text{rel},y} \right) \dot{x}. \end{aligned} \quad (34)$$

Thus,

$$\dot{h}_{\text{C3}} = L_f h_{\text{C3}} + L_g h_{\text{C3}} [a_f \quad a_l \quad \omega]^\top, \quad (35)$$

and the corresponding CBF condition is

$$L_f h_{\text{C3}} + L_g h_{\text{C3}} [a_f \quad a_l \quad \omega]^\top + \alpha (h_{\text{C3}}) \geq 0. \quad (36)$$

5) *Distance-Based Exponential CBF*: The distance-based barrier is

$$h_{\text{dist}} = \|\mathbf{p}_{\text{rel}}\|^2 - r^2. \quad (37)$$

Its first-order Lie derivatives are

$$L_f h_{\text{dist}} = 2\mathbf{p}_{\text{rel}}^\top \mathbf{v}_{\text{rel}}, \quad L_g h_{\text{dist}} = \mathbf{0}. \quad (38)$$

Therefore, the barrier has relative degree two with respect to the reduced-order dynamics. The second-order Lie derivatives are

$$L_f^2 h_{\text{dist}} = 2\|\mathbf{v}_{\text{rel}}\|^2, \quad (39)$$

and

$$L_g L_f h_{\text{dist}} = 2 \begin{bmatrix} p_{\text{rel},x} \cos \phi + p_{\text{rel},y} \sin \phi & -p_{\text{rel},x} \sin \phi + p_{\text{rel},y} \cos \phi & p_{\text{rel},y} \dot{x} - p_{\text{rel},x} \dot{y} \end{bmatrix}. \quad (40)$$

Hence,

$$\ddot{h}_{\text{dist}} = L_f^2 h_{\text{dist}} + L_g L_f h_{\text{dist}} \begin{bmatrix} a_f & a_l & \omega \end{bmatrix}^\top. \quad (41)$$

In particular, the coefficient associated with the yaw-rate input is

$$L_g L_f h_{\text{dist}}|_{\omega} = 2(p_{\text{rel},y} \dot{x} - p_{\text{rel},x} \dot{y}), \quad (42)$$

which is the expression used in the discussion of the yaw behavior.

The distance-based safety constraint is enforced using a second-order exponential CBF (ECBF):

$$L_f^2 h_{\text{dist}} + L_g L_f h_{\text{dist}} \begin{bmatrix} a_f & a_l & \omega \end{bmatrix}^\top + (\alpha_1 + \alpha_2) L_f h_{\text{dist}} + \alpha_1 \alpha_2 h_{\text{dist}} \geq 0, \quad (43)$$

where $\alpha_1 > 0$ and $\alpha_2 > 0$ are the ECBF gains.

B. Graph Encoding and Network Architecture

The high-level actor consists of a robot-centered graph attention encoder followed by an MLP policy head. During training, the input graph contains the robot, the goal, and at most 10 selected obstacles. Each node is represented as

$$\mathbf{n}_j = [\boldsymbol{\ell}_j^\top, (\mathbf{p}_j^b)^\top, r_j, (\mathbf{v}_j^b)^\top, m_j]^\top \in \mathbb{R}^9, \quad (44)$$

where $\boldsymbol{\ell}_j \in \mathbb{R}^3$ is a one-hot encoding identifying the robot, obstacle, or goal node; $\mathbf{p}_j^b \in \mathbb{R}^2$ and $\mathbf{v}_j^b \in \mathbb{R}^2$ denote the body-frame position and velocity, respectively; r_j is the node radius; and $m_j \in \{0, 1\}$ is the validity mask used for padded obstacle slots.

We employ robot-centered attention, in which only the robot-node embedding is computed. For each obstacle or goal node j , the edge from the robot node r is represented as

$$\mathbf{e}_{rj} = [\boldsymbol{\ell}_r^\top, \boldsymbol{\ell}_j^\top, (\mathbf{p}_j^b - \mathbf{p}_r^b)^\top, d_{rj}, (\mathbf{v}_j^b - \mathbf{v}_r^b)^\top]^\top \in \mathbb{R}^{11}, \quad (45)$$

where

$$d_{rj} = \|\mathbf{p}_j^b - \mathbf{p}_r^b\|_2 - (r_r + r_j) \quad (46)$$

denotes the minimum distance between the node boundaries. Thus, the 11-dimensional edge feature consists of the two 3-dimensional node-type labels, the 2-dimensional relative position, the scalar boundary distance, and the 2-dimensional relative velocity. The validity mask m_j is not passed through the edge MLP; it is used only to mask the attention logits of invalid padded nodes.

The attention mechanism first constructs an intermediate message $\mathbf{m}_{rj} = \psi_1(\mathbf{e}_{rj})$. The attention weight assigned to node j is computed as

$$\beta_{rj} = \frac{m_j \exp(\psi_2(\mathbf{m}_{rj}))}{\sum_k m_k \exp(\psi_2(\mathbf{m}_{rk}))}, \quad (47)$$

and the resulting robot-node embedding is

$$\mathbf{z}^{\text{GAT}} = \sum_j \beta_{rj} \psi_3(\mathbf{m}_{rj}) \in \mathbb{R}^{16}. \quad (48)$$

The detailed actor, critic, and frozen low-level policy architectures are summarized in Table IV. The actor concatenates the 16-dimensional graph embedding with the 13-dimensional local observation. The critic directly receives the corresponding noise-free privileged observation and uses a shared backbone with separate positive- and negative-value heads. The pretrained low-level locomotion policy remains fixed throughout high-level policy training.

TABLE IV: Network architectures used for humanoid navigation.

Component	Configuration
<i>Graph Attention Encoder</i>	
Graph input	Robot, goal, and up to 10 obstacle nodes
Node representation	$\mathbf{n}_j \in \mathbb{R}^9$
Edge representation	$\mathbf{e}_{r,j} \in \mathbb{R}^{11}$
Message network ψ_1	Linear($11 \rightarrow 64$) + ReLU, Linear($64 \rightarrow 16$)
Attention network ψ_2	Linear($16 \rightarrow 16$) + ReLU, Linear($16 \rightarrow 1$), masked softmax
Encoding network ψ_3	Linear($16 \rightarrow 64$) + ReLU, Linear($64 \rightarrow 16$)
GAT output	Attention-weighted robot embedding $\mathbf{z}^{\text{GAT}} \in \mathbb{R}^{16}$
<i>High-Level Actor</i>	
Actor input	Concat($\mathbf{z}^{\text{GAT}}[16], \mathbf{o}^{\text{local}}[13]$) = 29
Hidden layer 1	Linear($29 \rightarrow 256$) + ELU
Hidden layer 2	Linear($256 \rightarrow 128$) + ELU
Hidden layer 3	Linear($128 \rightarrow 64$) + ELU
Output layer	Linear($64 \rightarrow 6$)
Policy distribution	Three-dimensional tanh-squashed normal distribution
<i>Two-Head Critic</i>	
Critic input	Privileged observation: $12 \times 9 + 13 = 121$
Shared hidden layer 1	Linear($121 \rightarrow 256$) + ELU
Shared hidden layer 2	Linear($256 \rightarrow 256$) + ELU
Shared hidden layer 3	Linear($256 \rightarrow 128$) + ELU
Positive-value head	Linear($128 \rightarrow 1$)
Negative-value head	Linear($128 \rightarrow 1$)